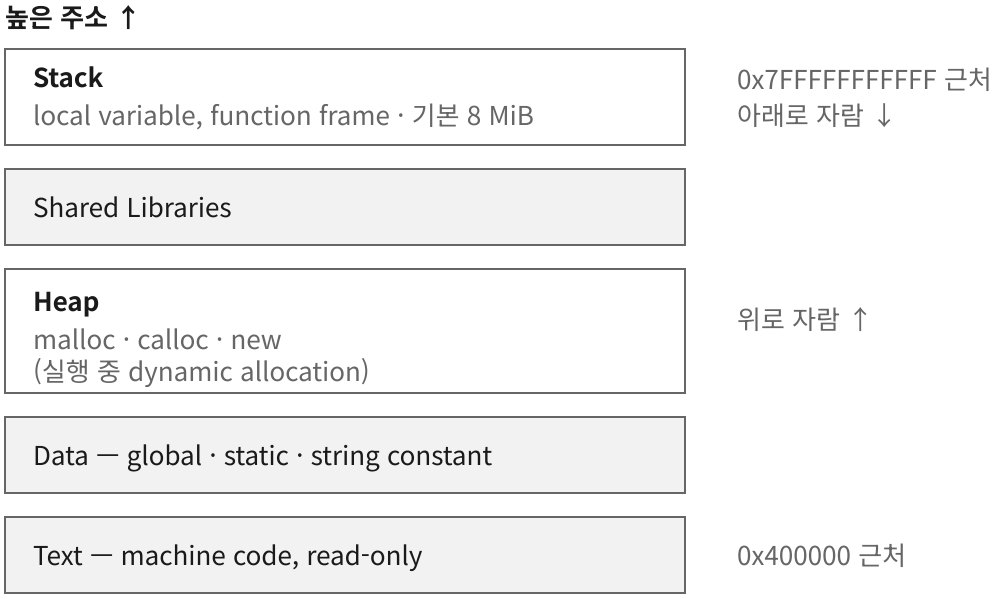
Lecture 8 · Advanced Topics

10/20 (화) · 슬라이드 "Lecture 8 Advanced Topics" + 수업 녹취 (14:04~14:53, ASR 누락 다수. 슬라이드 11~16의 disassembly walkthrough는 녹취가 거의 없어 슬라이드만으로 정리)

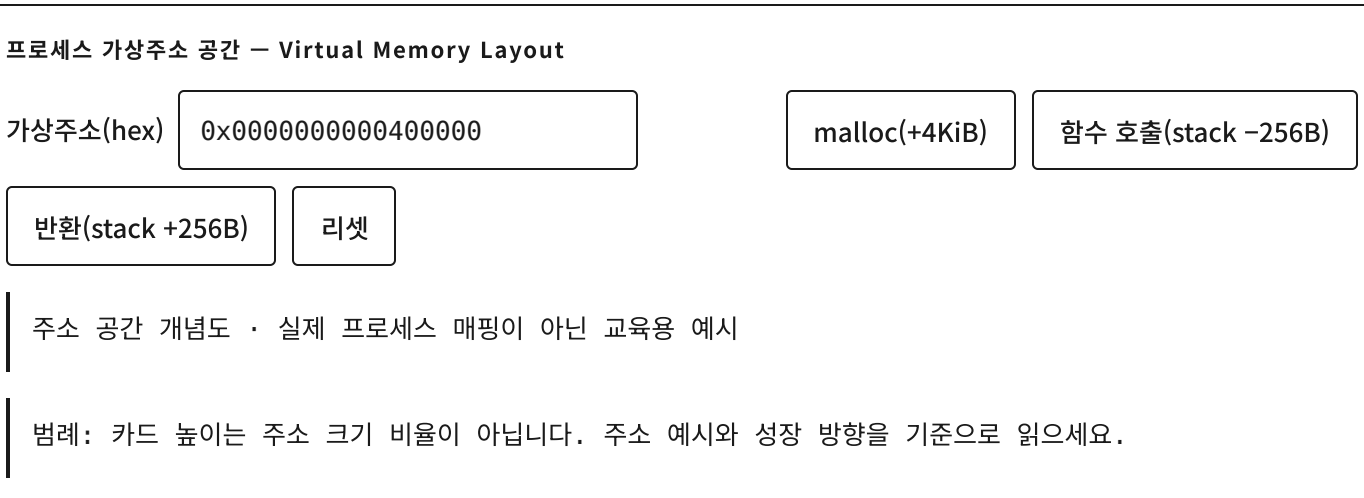
1 Memory layout

x86-64 address space의 다섯 영역



stack은 아래로, heap은 위로 자람 — 메모리 **할당 성장 방향**의 차이

(교정: 기존 "stack overflow와 heap overflow가 반대 방향" 설명은 부정확 — 반대인 것은 영역의 **성장 방향**뿐. stack buffer overflow는 buf에서 주소가 증가하는 쪽(saved return address 방향)으로 덮어쓰고, heap에서의 덮어쓰기 방향은 할당 배치·접근 방식에 따라 달라짐. CMU 15-213 대응 강의자료 기준, 교수님 강의 원본 미대조)



높은 주소 ↑ / 낮은 주소 ↓

커널 영역
0xFFFF800000000000 ~
사용자 모드 접근 불가

비정규 주소 간격
canonical hole
비정규 주소

스택(stack)
0x00007FFFFFFFE000 부근
↓ 낮은 주소 방향

mmap / 공유 라이브러리
0x00007F0000000000 부근
매핑 기반

미매핑 간격
주소 범위는 실행마다 다름

힙(heap)
0x0000000000800000 부근
↑ 높은 주소 방향

data / bss
0x0000000000600000 부근
전역·정적 데이터

text
0x0000000000400000 부근
읽기 + 실행
← 선택 주소

낮은 주소 / 보호 영역
0x0000000000000000 부근
NULL 등

선택 주소: 0x0000000000400000
영역 판정: text 코드 영역(예시, 읽기+실행)
Heap: 4096 B · Stack 사용량: 4096 B
사용자 영역 끝: 0x00007FFFFFFFFFFF · 첫 비정규 주소: 0x0000800000000000 · 커널 시작: 0xFFFF800000000000

Linux x86-64 주소 공간의 교육용 모델입니다. 카드 높이는 실제 주소 범위와 비례하지 않으며, 실제 매핑·권한·ASLR 배치는 운영체제와 프로세스에 따라 달라집니다.

array·heap·stack의 실제 주소

```
char big_array[1L<<24];   /* 16 MB global array */
char huge_array[1L<<31];  /* 2 GB global array */
p1 = malloc(1L<<28);      /* 256 MB */    p3 = malloc(1L<<32);  /* 4 GB */
p2 = malloc(1L<<8);       /* 256 B */     p4 = malloc(1L<<8);   /* 256 B */
```

영역	실행 예 주소
local variable (stack)	`0x7ffe4d3be87c`
p1, p3 (heap, 큰 할당)	`0x7f7262a1e010` , `0x7f7162a1d010`
p2, p4 (heap, 작은 할당)	`0x8359d010` , `0x8359d120`
big_array, huge_array (data)	`0x80601060` , `0x601060`
main, useless (text)	`0x40060c` , `0x400590`

- 주소가 서로 떨어져 있는 이유: heap은 일반적으로 높은 주소 방향으로, stack은 낮은 주소 방향으로 성장한다. Shared library는 mmap 등의 메커니즘을 통해 별도의 가상주소 영역에 매핑된다. 각 영역의 실제 주소와 배치는 운영체제, 프로세스 구성 및 ASLR 설정에 따라 달라진다.
- ASLR이 켜져 있으면 실행마다 stack 주소가 달라짐 (아래 Defenses)

2 Buffer overflow

struct_t 예제: array 범위를 넘으면

```
struct_t { int a[2]; double d; }      // a[0] offset 0, a[1] offset 4, d offset 8

fun(i) { s.d = 3.14; s.a[i] = 1073741824; return s.d; }
```

결과 해석 시 주의: a[2] 이상에 쓰는 것은 배열 범위를 벗어난 접근으로, C 언어에서 정의되지 않은 동작(undefined behavior)이다. 아래 수치는 특정 컴파일러·ABI·빌드에서 관찰되는 메모리 덮어쓰기 결과를 설명하기 위한 예시이며, C 표준이 보장하는 결과가 아니다.

일반적인 x86-64 little-endian 배치에서는 a[2]가 d의 낮은 4바이트를, a[3]가 d의 높은 4바이트를 덮어쓸 수 있다. 그러나 범위를 벗어난 접근이므로 컴파일러 최적화 등을 고려하면 실제 실행 결과를 일반화할 수 없다. fun(4)와 fun(6) 역시 정의되지 않은 동작이며, Segmentation fault 발생 여부나 손상되는 메모리 위치가 항상 동일한 것은 아니다.

호출	결과	이유
fun(0), fun(1)	3.14	정상 범위
fun(2)	3.1399998664856	a[2]가 d의 낮은 4byte를 덮음
fun(3)	2.00000061035156	a[3]이 d의 높은 4byte를 덮음 (낮은 4byte는 3.14 원래 값 유지)
fun(4)	3.14	d를 지난 칸 (이 실행에서는 무해)
fun(6)	Segmentation fault	critical state / return address까지 침범

결과가 시스템마다 다르다
struct padding과 stack 배치가 compiler·OS마다 다르기 때문에 같은 코드도 다른 결과가 나올 수 있음.

왜 큰 문제인가

- buffer overflow: array에 할당된 크기를 넘겨 쓰는 것. **기술적 원인 기준 보안 취약점 1위**
- 가장 흔한 형태: unchecked string length, 특히 stack의 bounded character array → **stack smashing**
- programmers가 같은 실수를 반복해 왔고, 최근 countermeasure로 공격이 어려워짐

취약한 gets()와 echo

```
char *gets(char *dest) { ... }   // EOF나 개행까지 읽고 *p++ = c, 마지막에 *p = '\0'
                                 // 길이를 지정하는 인자가 없음!
```

```
// 비슷하게 위험: strcpy, strcat (임의 길이 복사), scanf·fscanf·sscanf의 %s (길이 제한 없음)

void echo() {
    char buf[4];
    gets(buf);
    puts(buf);
}
```

echo의 stack frame — buf는 %rsp, return address는 %rsp+24

buf + padding (24byte) — %rsp
return address — %rsp+24

echo disassembly: sub $24,%rsp, %rsp를 gets 인자로 전달

입력 길이별 결과

- 23바이트 + null = 24 → buf와 padding을 꽉 채움 · 그대로 출력 (하지만 stack 일부를 이미 덮음)
- 24바이트 + null = 25 → return address 첫 byte를 0x00으로 덮음 · echo 출력은 정상, return 시 crash
- 25바이트 이상 → return address 여러 byte가 바뀜 · retq가 엉뚱한 주소로 점프 → Segmentation fault

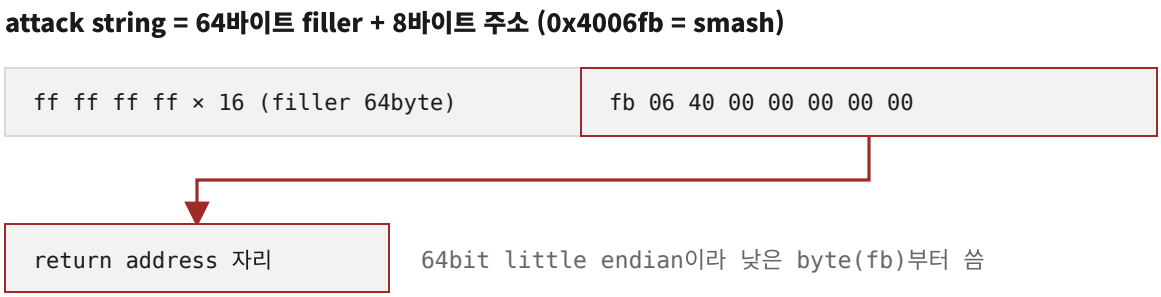
슬라이드의 23/24/25바이트 stack 그림을 하나로 정리

3 Stack smashing과 code injection

return address 덮어쓰기

```
void Q() {
    char buf[64];
    gets(buf);              // return address A를 다른 code S의 주소로 덮으면
}                           // Q가 ret할 때 A 대신 S로 점프

void smash() { printf("I've been smashed!\n"); exit; }  /* 슬라이드 표기 — 실제로는 exit(0)처
```



gets가 입력을 buf부터 순서대로 쓰므로, buf[64]를 지난 8바이트가 return address를 smash의 주소로 바꿈

이 그림은 buf의 시작점에서 return address까지의 오프셋이 64바이트라고 가정한 개념도이다. 실제 공격 문자열에 필요한 filler 길이는 해당 바이너리의 disassembly와 stack frame에서 확인한 반환 주소 오프셋으로 결정해야 한다. Saved %rbp나 alignment padding이 존재하면 필요한 filler 길이는 64바이트보다 길 수 있다. 따라서 64바이트 filler + 8바이트 주소는 해당 스택 배치에서 반환 주소 오프셋이 실제로 64바이트임을 확인한 경우에만 적용한다.

스택 버퍼 오버플로 — Stack Smashing

buf 크기 N 8 입력 문자열 HELLO

입력 길이(ASCII 예시) 5

Stack protector (canary) ☐ ASLR (개념 토글) ☑ NX (개념 토글) ☑ ret 실행

스택 메모리 — 낮은 주소에서 높은 주소 순서

1 · 지역 버퍼 buf[8] · 기준 주소 0x00007FFFFFFFE000

+00	+01	+02	+03	+04	+05	+06	+07
0x48	0x45	0x4c	0x4c	0x4f	0x00	–	–

2 · Saved %rbp (8바이트) · 기준 주소 0x00007FFFFFFFE008

+08	+09	+0A	+0B	+0C	+0D	+0E	+0F
–	–	–	–	–	–	–	–

3 · Return Address (8바이트) · 기준 주소 0x00007FFFFFFFE010

+10	+11	+12	+13	+14	+15	+16	+17
–	–	–	–	–	–	–	–

```
buf 크기: 8 B
입력 문자 수(Unicode 코드 포인트 기준): 5개
입력 UTF-8 바이트 수: 5 B
gets() 모델의 쓰기 길이: 6 B (UTF-8 5 B + 끝 NUL 1 B)
버퍼 경계 초과 바이트 수: 0 B

saved %rbp 침범: 아니오
return address 침범: 아니오
```

```
방어 기법 — Mitigations
Stack canary: 반환 전에 보호 값이 변경되었는지 검사합니다. 이 위젯에서는 8바이트 영역으로 단순화합니다.
ASLR: 개념상 활성 — 주소 배치를 무작위화하는 기법입니다. 토글은 개념 설명용이며 실제 주소나 침범 계산을 바꾸지 않습니다.
NX: 개념상 활성 — 비실행 메모리에서의 코드 실행을 제한합니다. 토글은 개념 설명용이며 실제 명령어 실행을 시뮬레이트하지 않습니다.

주의: ret 결과는 교육용 단순화 모델입니다. 실제 CPU의 ret, 프로세스 종료, 메모리 권한 및 운영체제의 방어 동작 전체를 재현하지 않습니다.

길이 슬라이더는 ASCII 'A' 반복 입력을 생성합니다. 텍스트 직접 입력 시 문자 수에 맞춰 슬라이더가 동기화됩니다. 한글 등 비ASCII 문자는 UTF-8에서 여러 바이트를 차지할 수 있으므로 버퍼 침범 계산은 문자 수가 아니라 바이트 수를 사용합니다.

핵심 방어는 입력 길이를 검사하는 안전한 코드입니다. 이 그림은 설명용 스택 배치이며 실제 컴파일러의 패딩, canary 및 프레임 포인터 구성은 다를 수 있습니다.
```

Code injection

- return address A를 **buf의 주소 B**로 덮고, buf 안에 실행할 machine code를 채움 → Q가 ret하면 %rip가 B로 이동해 stack 위 exploit code를 실행
- 성립 조건: stack이 executable할 것, B 주소를 예측 가능할 것 → 이후 non-executable stack과 ASLR로 어려워짐
- 실제 사례: 원격 machine에서 arbitrary code 실행 가능. Lab 4 Attack Lab에서 직접 실습

4 Defenses

code 수준(길이 제한 함수)

- `gets(buf)` → `fgets(buf, 4, stdin)` · `strcpy` → `strncpy` (널 종료를 보장하지 않으므로 별도 처리 필요) · `scanf %s` → fgets 또는 `%ns` (n은 널 문자 자리를 뺀 최대 문자 수 — char buf[4]라면 %3s)
- `gets(buf)` 대신 `fgets(buf, 4, stdin)`을 사용하면 입력 스트림에서 최대 3바이트를 읽고 끝에 NUL 문자를 추가한다. 개행 문자가 읽히면 그 바이트도 이 한도에 포함된다. 따라서 버퍼 크기가 4바이트일 때 저장 공간은 입력 데이터 최대 3바이트와 NUL 1바이트로 구성된다. UTF-8 문자열에서는 문자 수와 바이트 수가 다를 수 있으므로 버퍼 경계는 바이트 수를 기준으로 계산한다.
- code 수준 수정은 기본이지만 legacy code를 모두 고칠 수 없음 → system·compiler 수준 방어가 함께 필요

---

system 수준 1(ASLR)

- 프로그램 시작 시 stack에 임의 크기의 공간을 할당 → stack 주소를 통째로 이동시켜 buffer나 삽입 code의 시작 주소를 예측하기 어렵게 만듦
- 실행 예: local 주소가 0x7ffe4d3be87c, 0x7fff75a4f9fc, 0x7ffeadb7c80c, 0x7ffeaea2fdac, 0x7ffcd452017c로 매번 다름

---

system 수준 2(non-executable stack)

- 전통 x86: read-only 또는 writable만 표시. 읽을 수 있으면 실행 가능. x86-64: executable permission을 명시적으로 추가
- OS가 stack을 non-executable로 표시 → non-executable 영역 실행 시 CPU/OS가 fault
- **고전 code injection은 막지만 ROP는 막지 못함**

compiler(stack canary)

보호된 echo의 stack frame

buf — %rsp	canary — %rsp+8	return addr — %rsp+24

buffer 바로 뒤에 special value(canary)를 놓고, function exit 전에 check → 값이 바뀌면 something wrong
보호된 프로그램에 9바이트 입력 → *** stack smashing detected ***
GCC -fstack-protector : 스택 카나리 활성화 옵션 (기본 활성화 여부는 GCC 버전·배포판 빌드 설정에 따라 다름)

canary가 buf와 return address 사이를 지킴. overflow가 return address에 닿으려면 canary를 먼저 덮어야 함

```
mov    %fs:0x28,%rax     # canary 읽기
mov    %rax,0x8(%rsp)    # stack에 저장
xor    %eax,%eax         # register의 canary 복사본 지우기
...                      # gets 호출
mov    0x8(%rsp),%rax    # gets 후 canary 다시 읽기
xor    %fs:0x28,%rax     # 원본과 비교
je     .L9               # 같으면 정상 return
call   __stack_chk_fail  # 다르면 abort
```

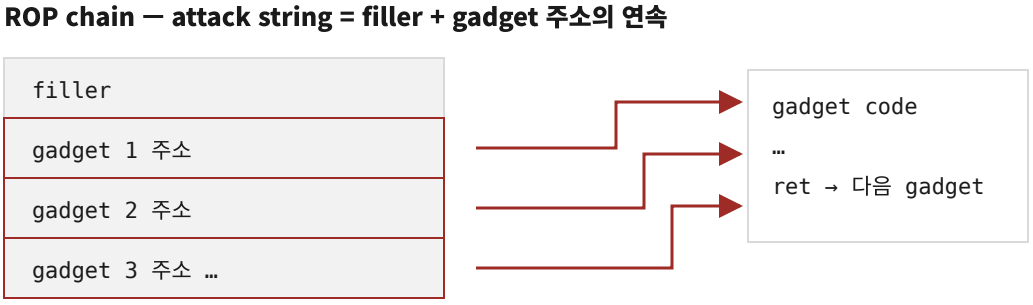
off-by-one과 canary의 한계

- canary 최하위 byte는 0x00인 경우가 많음 → 1바이트 null로 그 byte를 다시 0x00으로 덮으면 검사 통과 가능. 슬라이드 표현: benign corruption (무해한 손상). 프로그래머가 silent off-by-one 오류를 남길 수 있음
- canary는 return address를 덮는 공격을 막는 데 초점. buffer 앞쪽 local variable 손상까지 막지는 않음

5 Return-Oriented Programming (ROP)

- 배경: stack randomization + non-executable stack → code injection이 어려워짐. 대안: **이미 존재하는 code/library code를 재사용**해 fragment(gadget)를 이어 붙임
- Gadget은 이미 존재하는 실행 가능한 코드 안에서 찾아낸 명령어 조각으로, 보통 ret(opcode 0xc3)으로 끝난다. Gadget의 코드가 실행 가능하다는 사실과 그 가상주소가 고정된다는 사실은 다르다. PIE와 ASLR이 코드 매핑 주소를 무작위화하면 실행마다 gadget 주소가 달라질 수 있으며, 공격에 사용하려면 해당 주소를 알아내는 추가 정보가 필요할 수 있다.

> •예: ab_plus_c gadget (0x4004d4): `lea (%rdi,%rdx,1),%rax; ret` — rax ← rdi + rdx. gadget 주소가 0x4004d4이므로 그 앞 0x4004d0의 imul %rsi,%rdi는 실행되지 않음 · setval gadget (0x4004dc): 바이트열 c7 07 d4 48 89 c7 안에 48 89 c7 (movq %rax,%rdi)이 숨어 있고 뒤에 c3 (ret)이 이어짐. 중간 지점부터 실행하면 gadget이 됨



여러 gadget이 stack을 위로(주소 증가 방향으로) 침범하며 이어짐

(교정: 각 gadget의 ret이 stack에서 주소를 pop하며 %rsp가 증가 → CMU 15-213 대응 자료도 "Multiple gadgets will corrupt stack upwards". 교수님 강의 원본 미대조)

무엇을 우회하고 무엇을 못 우회하나

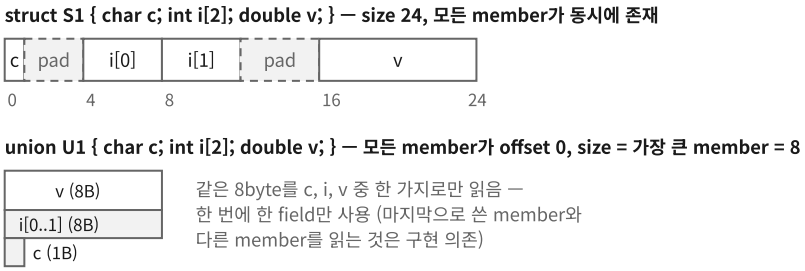
ROP가 각 방어 기법에 미치는 영향은 다음과 같다.

- **NX**: 기존의 실행 가능한 코드 조각을 재사용하므로, stack의 코드를 직접 실행하지 않고도 공격을 구성할 수 있다.
- **ASLR**: gadget의 주소를 알아야 한다. 실행 파일이나 라이브러리의 코드 주소가 고정되어 있으면 우회가 가능할 수 있으며, 주소가 무작위화되면 주소 유출 등 추가 정보가 필요할 수 있다.
- **Stack canary**: 일반적인 ROP는 반환 주소를 덮어써야 하므로 canary 검사가 추가 장벽이 된다. Canary 값이 유출되는 등 조건이 달라지면 우회 가능성이 생긴다.

(CMU 대응 자료: "Does not overcome stack canaries". CMU 15-213 대응 강의자료 기준, 교수님 강의 원본 미대조)

6 Union

struct와 union의 배치



같은 member 목록인데 struct는 24byte, union은 8byte

bit pattern 재해석

```
union { float f; unsigned u; };

float bit2float(unsigned u0) { u = u0; return f; }  // u에 bit pattern을 쓰고 f로 읽음 (슬라
unsigned float2bit(float f0) { f = f0; return u; }  // f에 값을 쓰고 u로 읽음
```

- `(float) u` / `(unsigned) f`는 **값 변환** · union은 **같은 bit의 재해석**. 둘은 완전히 다름
- IEEE 754 float의 sign·exponent·fraction을 확인할 때 사용

7 Byte ordering

- short/int/long: memory에 2/4/8개의 연속 byte (long 8byte는 Linux x86-64 LP64 기준 — Windows x64는 4byte). 어느 byte가 가장 낮은 주소에 오는가가 endian

---

Big endian
most significant byte가 가장 낮은 주소 (SPARC)
Little endian
least significant byte가 가장 낮은 주소 (Intel x86) — 강의 녹화본 표현: "lll 법칙" (LSB in Lowest address)
Bi-endian
설정에 따라 둘 다 (ARM)

- machine 사이에 binary data를 교환할 때 문제가 됨

union { unsigned char c[8]; short s[4]; int i[2]; long l[1]; } dw — dw.c[0..7] = 0xf0..0xf7

f0	f1	f2	f3	f4	f5	f6	f7

c[0] (낮은 주소) → c[7] (높은 주소)

Little endian (x86)

s[0]=0xf1f0 · i[0]=0xf3f2f1f0

l[0]=0xf7f6f5f4f3f2f1f0

Big endian (SPARC)

s[0]=0xf0f1 · i[0]=0xf0f1f2f3

l[0]=0xf0f1f2f3f4f5f6f7

32-bit machine은 long이 4byte라 l[0] 값이 64-bit와 다름

같은 byte열을 little endian은 뒤집어 읽고, big endian은 그대로 읽음

바이트 순서 — Little / Big Endian

정수(hex 또는 10진) 0x01234567

32비트(int) | 16비트(short) | 문자열(ASCII)

Little-endian — x86-64

낮은 주소 (Low Address) → 높은 주소 (High Address)

주소 +0 0x67	주소 +1 0x45	주소 +2 0x23	주소 +3 0x01

Big-endian

낮은 주소 (Low Address) → 높은 주소 (High Address)

주소 +0 0x01	주소 +1 0x23	주소 +2 0x45	주소 +3 0x67

```
입력값: 0x01234567
표시 폭: 4바이트
해당 폭으로 해석한 값: 0x01234567

Little-endian: 0x67 0x45 0x23 0x01
Big-endian: 0x01 0x23 0x45 0x67

화살표는 낮은 메모리 주소에서 높은 메모리 주소로 증가하는 방향입니다.
Little-endian은 최하위 바이트(LSB)를 낮은 주소에 저장합니다.
Big-endian은 최상위 바이트(MSB)를 낮은 주소에 저장합니다.
바이트 저장 순서는 달라도 같은 폭과 해석 규칙을 사용하면 원래 정수 값을 복원할 수 있습니다.
```

8 시험 포인트

- memory layout: Text 0x400000 근처 · Data (global·static·string constant) · Heap (malloc, 위로 자람) · Shared Libraries · Stack 0x7FFFFFFFFFFF 근처 (기본 8 MiB, 아래로 자람)
- struct_t: a[0] offset 0, a[1] offset 4, d offset 8 → a[2]부터 d를 덮음 · 결과는 시스템마다 다를 수 있음 (padding·stack 배치가 compiler·OS에 따라 다름)
- echo frame: buf %rsp, return address %rsp+24 · 24바이트 입력 + null이 return address 첫 byte를 덮음 · gets/strcpy/strcat/%s는 길이 제한이 없어 위험
- 방어, code 수준: fgets·strncpy (널 종료 미보장 — 별도 처리 필요)·%ns (n은 buffer 크기-1)
- 방어, system 수준: ASLR (stack 주소를 매번 이동), non-executable stack (고전 injection은 막지만 ROP는 못 막음)
- 방어, compiler 수준: stack canary는 GCC의 -fstack-protector 계열 옵션으로 활성화할 수 있다. 기본 활성화 여부는 GCC 버전, 배포판 및 빌드 설정에 따라 달라진다. 대표적인 Linux x86-64 구현에서는 %fs:0x28에서 보호 값을 읽고, 검사 실패 시 __stack_chk_fail을 호출한다.
- canary 한계: 최하위 byte 0x00인 경우가 많아 off-by-one null 덮기는 통과 가능 · buffer 앞쪽 local variable 손상은 못 막음
- ROP gadget: ret (0xc3)으로 끝나는 기존 실행 코드 조각을 연결한다.
- NX와 ASLR: ROP는 non-executable stack을 우회할 수 있다. ASLR 우회는 코드 주소가 고정되어 있거나 주소 무작위화를 극복할 정보가 확보된 경우에 가능하다.
- Stack canary: 일반적인 ROP는 반환 주소 변조가 필요하므로 canary가 장벽이 된다. Canary 값이 유출되는 등의 조건에서는 우회 가능성이 생긴다.
- struct S1: offset c@0, i[0]@4, i[1]@8, v@16, size 24 (모든 member가 동시에 존재) · union U1: 모두 offset 0, size 8 (한 번에 한 field만 — 다른 member를 읽는 것은 구현 의존)
- union은 같은 bit의 재해석, cast는 값 변환. bit2float/float2bit에 사용
- endian: dw.c = f0..f7일 때 little endian s[0]=0xf1f0, i[0]=0xf3f2f1f0, l[0]=0xf7f6f5f4f3f2f1f0 · big endian은 그대로 0xf0f1, 0xf0f1f2f3, 0xf0f1f2f3f4f5f6f7

9 연습문제

문제 1. struct_t에서 fun(2)와 fun(3)이 3.14가 아닌 값이 나오는 이유를 offset으로 설명하라.

정답 보기

① 배치는 a[0] offset 0, a[1] offset 4, d offset 8 → a[2]는 정확히 d가 시작하는 자리

② a[2] = 1073741824를 쓰면 d의 첫 4byte(낮은 byte)가 바뀜 → 3.14가 3.13999986648856이 됨

③ a[3]은 d의 높은 4byte를 덮음 (낮은 4byte는 3.14 원래 값 유지) → d = 2.00000061035156

최종 답 **a[2]는 d의 낮은 4byte, a[3]은 d의 높은 4byte를 덮기 때문**

문제 2. echo()의 buf[4]에 24바이트를 넣으면 return address가 어떻게 바뀌는가?

정답 보기

① frame: buf 4byte + padding 20byte = 24byte, return address는 %rsp+24

② 24번째 byte까지 채우고 gets가 붙이는 null(25번째 byte)이 return address의 첫 byte를 0x00으로 덮음

③ 원래 return address 0x4006f6 → 0x400600

④ 24문자는 정상 echo 출력됨. 그 후 retq가 0x400600으로 점프해 segmentation fault

(교정: 출력이 깨지는 것이 아니라 출력은 정상이고 return 시에만 crash. CMU 15-213 대응 자료의 같은 예 기준, 교수님 강의 원본 미대조)

최종 답 **0x4006f6 → 0x400600 (첫 byte가 0x00으로 덮임)**

문제 3. dw.c[0..7] = 0xf0..0xf7일 때 little endian과 big endian의 s[0], i[0], l[0]을 구하라.

정답 보기

① Little endian (낮은 자리 byte가 낮은 주소): s[0] = 0xf1f0, i[0] = 0xf3f2f1f0, l[0] = 0xf7f6f5f4f3f2f1f0

② Big endian (높은 자리 byte가 낮은 주소): s[0] = 0xf0f1, i[0] = 0xf0f1f2f3, l[0] = 0xf0f1f2f3f4f5f6f7

최종 답

little: 0xf1f0 / 0xf3f2f1f0 / 0xf7f6f5f4f3f2f1f0 · big: 0xf0f1 / 0xf0f1f2f3 / 0xf0f1f2f3f4f5f6f7